

# مقایسه روش‌های یادگیری گروهی (Ensemble Learning)

## Boosting در برابر Bagging

پیاده‌سازی و تحلیل تطبیقی روی مجموعه‌داده Carseats (زبان R)

آیلین رحمانی‌پور

### ۱. مقدمه و هدف پروژه

هدف این پروژه، مقایسه‌ی عملکرد یک مدل پایه (درخت تصمیم ساده) با سه روش یادگیری گروهی AdaBoost، Bagging و Gradient Boosting در یک مسئله‌ی دسته‌بندی دودویی است. برای این منظور از مجموعه‌داده‌ی Carseats استفاده شده و عملکرد چهار مدل با معیارهای Accuracy، AUC، Sensitivity، Specificity و Kappa با یکدیگر مقایسه شده‌اند.

### ۲. آماده‌سازی داده

در مجموعه‌داده‌ی Carseats، متغیر Sales عددی و پیوسته است؛ بنابراین برای تبدیل مسئله به یک مسئله‌ی دسته‌بندی دودویی، متغیر جدید High با استفاده از تابع ifelse() ساخته شد: اگر فروش بیشتر از ۸ واحد باشد «Yes»، در غیر این صورت «No». سپس ستون Sales از داده حذف شد تا از Data Leakage جلوگیری شود.

توزیع کلاس‌ها (No=۵۹٪، Yes=۴۱٪) نشان می‌دهد داده نسبتاً متوازن است، بنابراین معیار قابل‌اعتمادی برای ارزیابی مدل‌ها خواهد بود. داده به نسبت ۷۰٪ آموزش و ۳۰٪ آزمون (با seed=۱۲۳) تقسیم شد.

### ۳. روش‌ها و پیاده‌سازی

#### ۳.۱ مدل پایه: درخت تصمیم

قبل از سنجش روش‌های پیشرفته‌تر (Bagging و Boosting)، لازم بود یک معیار مبنا (Baseline) وجود داشته باشد تا بتوان میزان بهبود واقعی هر روش را نسبت به یک مدل ساده اندازه گرفت. با تابع rpart() از پکیج rpart یک درخت تصمیم استاندارد

روی داده‌ی آموزش ساخته شد و با تابع `confusionMatrix()` از پکیج `caret`، ماتریس درهم‌ریختگی و معیارهای تکمیلی (Sensitivity، Specificity، Kappa و...) استخراج شدند.

```
base_model <- rpart(High ~ ., data = train_data, method = "class")
```

## ۳.۲ Bagging

Bagging یعنی ساخت چند مدل مستقل روی نمونه‌های `Bootstrap` و میانگین‌گیری از آن‌ها برای کاهش `Variance`. در R این کار با تابع `randomForest()` پیاده می‌شود؛ اما نکته‌ی مهم اینجاست: `Random Forest` به‌طور پیش‌فرض در هر تقسیم فقط زیرمجموعه‌ای تصادفی از متغیرها را در نظر می‌گیرد، در حالی که Bagging باید همه‌ی متغیرها را در هر تقسیم ببیند. برای همین `mtry = ncol(train_data) - 1` تنظیم شد؛ یعنی همه‌ی متغیرها (به‌جز خود متغیر هدف) در دسترس‌اند. این دقیقاً همان چیزی است که Bagging را از یک `Random Forest` واقعی متمایز می‌کند.

```
bag_model <- randomForest(High ~ ., data = train_data,
                           mtry = ncol(train_data) - 1,
                           ntree = 2000, importance = TRUE)
```

## ۳.۳ AdaBoost

مدل `AdaBoost` در تابع `gbm()` پیاده شد. مقدار `interaction.depth = 1` عمدتاً انتخاب شد تا هر مدل پایه یک `Decision Stump` باشد؛ یعنی همان تعریف کلاسیک `AdaBoost` که در آن مدل‌های بسیار ساده، مرحله‌به‌مرحله و با وزن‌دهی به نمونه‌های سخت، ترکیب می‌شوند. تنظیم `cv.folds = 5` همراه با `gbm.perf(method = "cv")` تعداد بهینه‌ی درخت‌ها را با اعتبارسنجی متقابل انتخاب می‌کند، به‌جای اینکه عددی دلخواه باشد.

```
ada_model <- gbm(High ~ ., data = train_gbm,
                 distribution = "adaboost", n.trees = 2000,
                 interaction.depth = 1, shrinkage = 0.01, cv.folds = 5)
```

## ۳.۴ Gradient Boosting

به‌جای وزن‌دهی نمونه‌ها (مانند `AdaBoost`)، در `Gradient Boosting` هر مدل جدید روی خطای باقی‌مانده `(Residual/Gradient)` مدل قبلی آموزش می‌بیند. برخلاف `AdaBoost`، اینجا `interaction.depth = 3` تنظیم شد تا هر

مدل پایه کمی پیچیده‌تر از یک Stump باشد و بتواند تعامل بین متغیرها را نیز در نظر بگیرد. مانند قبل،  $cv.folds = 5$  و  $gbm.perf()$  تعداد بهینه‌ی درخت‌ها را با اعتبارسنجی متقابل انتخاب می‌کند تا از Overfitting جلوگیری شود.

```
gbm_model <- gbm(High ~ ., data = train_gbm,
  distribution = "bernoulli", n.trees = 2000,
  interaction.depth = 3, shrinkage = 0.01, cv.folds = 5)
```

## ۴. نتایج به تفکیک مدل

### ۴.۱ مدل پایه (Decision Tree)

| واقعی: Yes | واقعی: No |               |
|------------|-----------|---------------|
| ۱۹         | ۵۶        | پیش‌بینی: No  |
| ۳۰         | ۱۵        | پیش‌بینی: Yes |

| مقدار  | معیار       |
|--------|-------------|
| ۰.۷۱۶۷ | Accuracy    |
| ۰.۷۸۳۱ | AUC         |
| ۰.۶۱۲۲ | Sensitivity |
| ۰.۷۸۸۷ | Specificity |
| ۰.۴۰۶۱ | Kappa       |

از ۱۲۰ نمونه‌ی آزمون، مدل پایه ۳۴ مورد را اشتباه پیش‌بینی کرد (۱۹ فروشگاه پر فروش نادیده گرفته شد و ۱۵ فروشگاه کم‌فروش به‌اشتباه Yes در نظر گرفته شد).

دقت ۷۱.۶۷٪ طبق  $P\text{-Value}=0.003$  به‌طور معنادار بهتر از حدس ساده‌ی کلاس اکثریت (۵۹.۱۷٪) است.

تفاوت Sensitivity (۶۱.۲۲٪) و Specificity (۷۸.۸۷٪) نشان می‌دهد مدل در تشخیص فروشگاه‌های کم‌فروش بهتر از پر فروش عمل می‌کند

مقدار  $Kappa=0.4061$  نشان‌دهنده‌ی توافق متوسط با واقعیت (فراتر از شانس) و  $AUC=0.7831$  نشان‌دهنده‌ی قدرت تفکیک قابل قبول اما نه چندان قوی است. به طور کلی، مدل پایه عملکرد متوسطی دارد و به خصوص در شناسایی کلاس Yes ضعیف است؛ همین ضعف، نقطه‌ی مقایسه برای سنجش میزان بهبود روش‌های بعدی خواهد بود.

## ۴.۲ Bagging

| واقعی: Yes | واقعی: No |               |
|------------|-----------|---------------|
| ۱۲         | ۶۵        | پیش‌بینی: No  |
| ۳۷         | ۶         | پیش‌بینی: Yes |

| مقدار  | معیار       |
|--------|-------------|
| ۰.۸۵۰۰ | Accuracy    |
| ۰.۸۹۹۴ | AUC         |
| ۰.۷۵۵۱ | Sensitivity |
| ۰.۹۱۵۵ | Specificity |
| ۰.۶۸۳۶ | Kappa       |

از ۱۲۰ نمونه، ۱۸ مورد اشتباه پیش‌بینی شد (۱۲ فروشگاه پر فروش نادیده گرفته شد، ۶ فروشگاه کم فروش به اشتباه Yes زده شد). دقت ۸۵.۰۰٪ به طور معنادار بهتر از حدس ساده است (P-Value بسیار کوچک:  $10^{-10} \times 8.13$ ).

نسبت به مدل پایه، هر دو معیار (Sensitivity (۷۵.۵۱٪) و Specificity (۹۱.۵۵٪) بهبود چشمگیری یافته‌اند. Kappa از ۰.۴۰۶۱ به ۰.۶۸۳۶ جهش کرده که نشان‌دهنده‌ی توافق قوی با واقعیت است، و  $AUC=0.8994$  قدرت تفکیک قوی مدل را نشان می‌دهد. میانگین‌گیری از ۲۰۰۰ درخت مستقل، واریانس مدل پایه را به شدت کاهش داده و پایداری و دقت را هم‌زمان بالا برده است.

## ۴.۳ AdaBoost

| واقعی: Yes | واقعی: No |               |
|------------|-----------|---------------|
| ۱۰         | ۶۷        | پیش‌بینی: No  |
| ۳۹         | ۴         | پیش‌بینی: Yes |

| مقدار  | معیار       |
|--------|-------------|
| ۰.۸۸۳۳ | Accuracy    |
| ۰.۹۳۳۹ | AUC         |
| ۰.۷۹۵۹ | Sensitivity |
| ۰.۹۴۳۷ | Specificity |
| ۰.۷۵۳۹ | Kappa       |

از ۱۲۰ نمونه، تنها ۱۴ مورد اشتباه پیش‌بینی شد (۱۰ فروشگاه پر فروش نادیده گرفته شد و فقط ۴ فروشگاه کم‌فروش به اشتباه Yes زده شد). دقت ۸۸.۳۳٪ بهترین Accuracy در بین همه‌ی مدل‌ها تا این مرحله است (برابر با Gradient Boosting).

تفاوت Sensitivity (۷۹.۵۹٪) و Specificity (۹۴.۳۷٪) نشان می‌دهد مدل در تشخیص No تقریباً بی‌خطاست، اما در تشخیص Yes کمی محتاط‌تر عمل می‌کند.  $Kappa=0.7539$  توافق قوی با واقعیت را نشان می‌دهد و  $AUC=0.9339$  بالاترین قدرت تفکیک در بین هر چهار مدل است. وزن‌دهی مجدد به نمونه‌های اشتباه پیش‌بینی شده باعث شده مدل به شدت در تشخیص No دقیق شود و بالاترین AUC پروژه را به دست آورد.

#### ۴.۴ Gradient Boosting

| واقعی: Yes | واقعی: No |               |
|------------|-----------|---------------|
| ۹          | ۶۶        | پیش‌بینی: No  |
| ۴۰         | ۵         | پیش‌بینی: Yes |

| مقدار  | معیار       |
|--------|-------------|
| ۰.۸۸۳۳ | Accuracy    |
| ۰.۹۲۱۵ | AUC         |
| ۰.۸۱۶۳ | Sensitivity |
| ۰.۹۲۹۶ | Specificity |
| ۰.۷۵۵۵ | Kappa       |

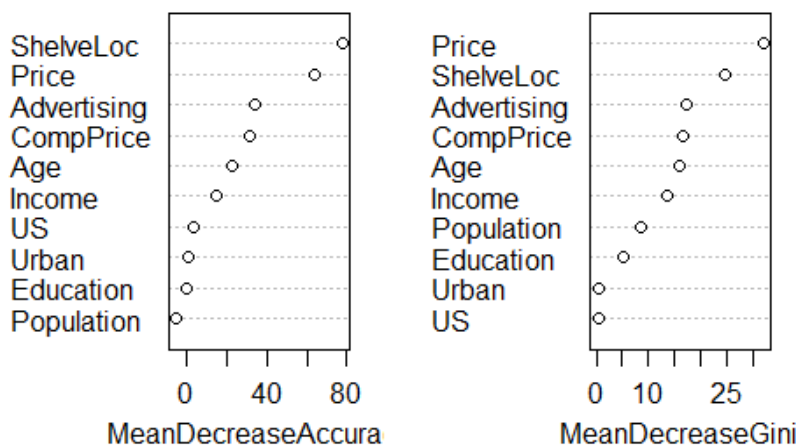
از ۱۲۰ نمونه، ۱۴ مورد اشتباه پیش‌بینی شد (۹ فروشگاه پر فروش نادیده گرفته شد، ۵ فروشگاه کم‌فروش به اشتباه Yes زده شد) دقیقاً برابر با AdaBoost از نظر Accuracy (۸۸.۳۳٪).

نسبت به AdaBoost، مدل در تشخیص Yes ( $\text{Sensitivity}=81.63\%$  در برابر  $79.59\%$ ) کمی بهتر عمل کرده، اما در تشخیص No ( $\text{Specificity}=92.96\%$  در برابر  $94.37\%$ ) کمی ضعیف‌تر بوده است.  $\text{Kappa}=0.7555$  تقریباً برابر با AdaBoost ( $0.7539$ ) است و توافق بسیار قوی با واقعیت را نشان می‌دهد.  $\text{AUC}=0.9215$  قوی است، هرچند کمی پایین‌تر از AdaBoost ( $0.9339$ ). برازش روی خطای باقی‌مانده (Residual) باعث شده مدل در شناسایی فروشگاه‌های پر فروش (Yes) کمی موفق‌تر از AdaBoost عمل کند، هرچند از نظر Accuracy و Kappa تقریباً هم‌سطح هستند.

## ۵. اهمیت متغیرها

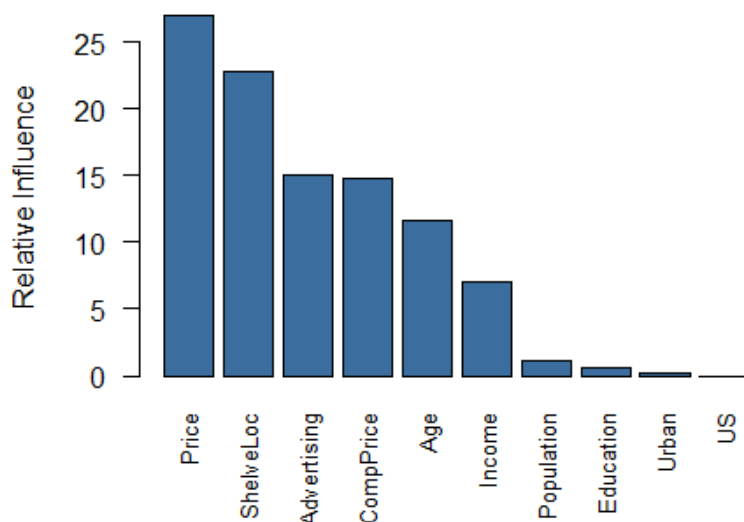
یکی از مزیت‌های مهم روش‌های Ensemble نسبت به سایر مدل‌های Black-box این است که همچنان می‌توانند نشان دهند کدام متغیرها بیشترین نقش را در پیش‌بینی داشته‌اند. این تحلیل کمک می‌کند نتیجه‌ی مدل را از نظر کسب‌وکاری هم تفسیر کنیم، نه فقط از نظر آماری.

### Variable Importance - Bagging



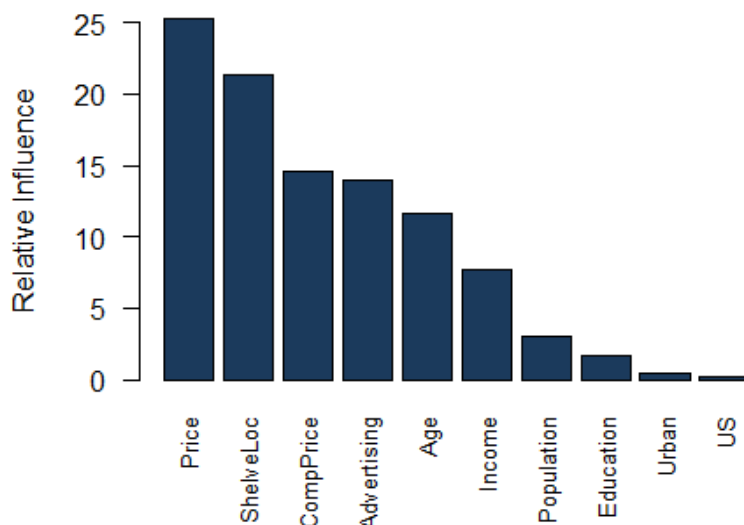
نمودار ۱ — اهمیت متغیرها بر اساس مدل Bagging (Random Forest)

## Variable Importance - AdaBoost



نمودار ۲ — اهمیت متغیرها بر اساس مدل *AdaBoost*

## Variable Importance - Gradient Boosting



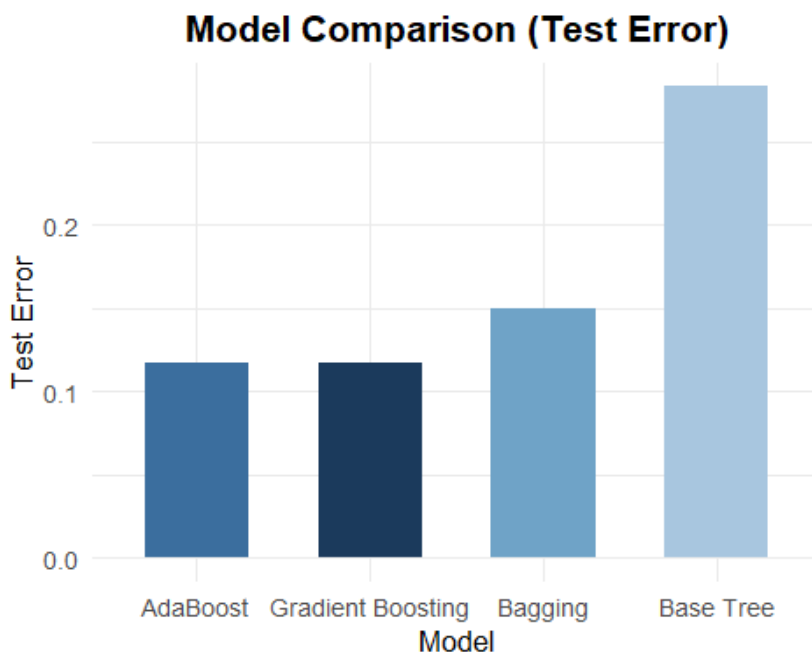
نمودار ۳ — اهمیت متغیرها بر اساس مدل *Gradient Boosting*

در هر سه روش (AdaBoost، Bagging و Gradient Boosting)، دو متغیر *ShelveLoc* (موقعیت قفسه) و *Price* (قیمت) به‌طور پیوسته رتبه‌ی اول و دوم اهمیت را دارند. رتبه‌های بعدی *Advertising*، *CompPrice* و *Age* در هر سه مدل در جایگاه سوم تا پنجم قرار می‌گیرند، هرچند ترتیب دقیق‌شان کمی جابه‌جا می‌شود. متغیرهای *Urban*، *US* و *Education* در هر سه مدل تقریباً نزدیک به صفر هستند؛ یعنی نقش عملی در پیش‌بینی فروش بالا/پایین ندارند.

اهمیت این توافق در این است که وقتی سه الگوریتم کاملاً متفاوت (یکی مبتنی بر Bootstrap مستقل، دو تای دیگر مبتنی بر یادگیری مرحله‌ای) به یک نتیجه‌ی مشابه می‌رسند، این خود شاهد قوی است که الگویی واقعی و پایدار در داده وجود دارد، نه یک یافته‌ی تصادفی مختص یک مدل خاص.

## ۶. مقایسه نهایی مدل‌ها

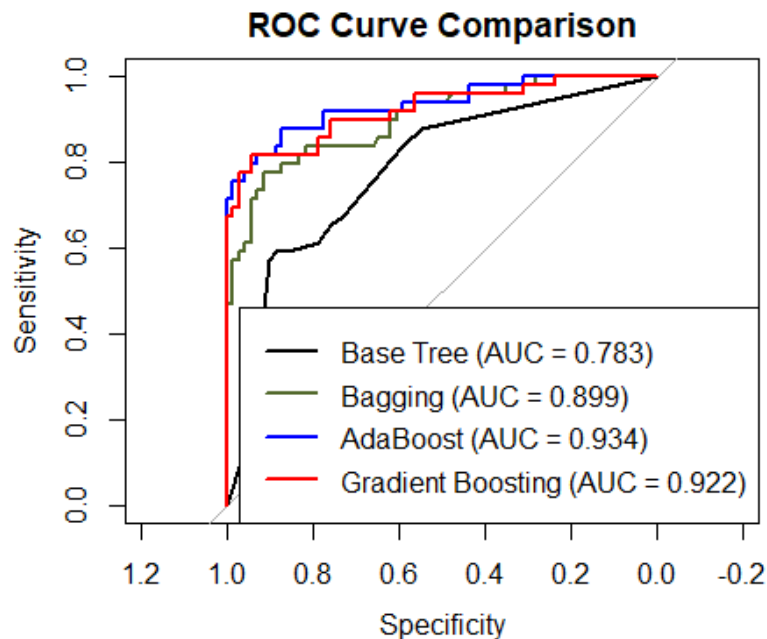
| مدل               | Accuracy | Test Error | AUC    | Sensitivity | Specificity | Kappa  |
|-------------------|----------|------------|--------|-------------|-------------|--------|
| مدل پایه          | ۰.۷۱۶۷   | ۰.۲۸۳۳     | ۰.۷۸۳۱ | ۰.۶۱۲۲      | ۰.۷۸۸۷      | ۰.۴۰۶۱ |
| Bagging           | ۰.۸۵۰۰   | ۰.۱۵۰۰     | ۰.۸۹۹۴ | ۰.۷۵۵۱      | ۰.۹۱۵۵      | ۰.۶۸۳۶ |
| AdaBoost          | ۰.۸۸۳۳   | ۰.۱۱۶۷     | ۰.۹۳۳۹ | ۰.۷۹۵۹      | ۰.۹۴۳۷      | ۰.۷۵۳۹ |
| Gradient Boosting | ۰.۸۸۳۳   | ۰.۱۱۶۷     | ۰.۹۲۱۵ | ۰.۸۱۶۳      | ۰.۹۲۹۶      | ۰.۷۵۵۵ |



نمودار ۴ — مقایسه‌ی خطای آزمون (*Test Error*) چهار مدل

نمودار به‌وضوح افت پلکانی خطا را نشان می‌دهد: مدل پایه بیشترین خطا (۲۸.۳۳٪) را دارد، Bagging آن را تقریباً به نصف می‌رساند (۱۵٪)، و دو روش Boosting کمترین خطا (۱۱.۶۷٪) را ثبت می‌کنند. این الگوی بصری دقیقاً همان چیزی است که تئوری پیش‌بینی می‌کند: هرچه از مدل ساده به سمت روش‌های Ensemble پیچیده‌تر حرکت می‌کنیم، خطا به‌طور سیستماتیک کاهش می‌یابد. تفاوت ناچیز بین AdaBoost و Gradient Boosting نشان می‌دهد در این دیتاست خاص، هر دو مکانیزم Boosting (وزن‌دهی نمونه در برابر برازش روی Residual) به یک سطح عملکرد رسیده‌اند.





نمودار ۵ — مقایسه‌ی منحنی‌های ROC چهار مدل

جدول و نمودار Test Error فقط عملکرد مدل را در یک آستانه‌ی ثابت (۰.۵) نشان می‌دهند. منحنی ROC عملکرد مدل را در همه‌ی آستانه‌های ممکن ترسیم می‌کند و تصویر کامل‌تری از قدرت واقعی تفکیک‌پذیری هر مدل ارائه می‌دهد؛ هرچه منحنی به گوشه‌ی بالا-چپ نمودار نزدیک‌تر باشد، مدل قوی‌تر است. منحنی AdaBoost بیشترین نزدیکی به این گوشه را دارد و بالاترین AUC (۰.۹۳۳۹) را ثبت کرده، در حالی که منحنی مدل پایه پایین‌ترین و نزدیک‌ترین به خط قطری است. این نمودار به‌صورت بصری همان نتیجه‌ای را تأیید می‌کند که در جدول عددی دیده شد: روند پیشرفت پلکانی از مدل پایه تا Boosting.

## ۷. جمع‌بندی

مدل پایه ضعیف‌ترین عملکرد را دارد. Bagging با کاهش Variance آن را به‌طور محسوس بهبود می‌دهد (جهش Kappa از ۰.۴۰ به ۰.۶۸)، و دو روش Boosting با کاهش Bias فراتر از Bagging می‌روند و به بالاترین AUC و Accuracy پروژه می‌رسند. AdaBoost و Gradient Boosting دقیقاً همان Accuracy را دارند، اما نوع خطایشان فرق می‌کند: AdaBoost در تشخیص No محتاط‌تر و دقیق‌تر است، در حالی که Gradient Boosting در شناسایی Yes کمی موفق‌تر عمل کرده است. این یعنی هیچ‌کدام «مطلقاً برتر» نیستند و انتخاب مدل نهایی به هدف کاربردی بستگی دارد.